

Neo4j 图数据库通俗指南

零基础也能看懂的 Neo4j 入门

 = 该术语在 [01_AI_terminology_explanations.md](#) 中有独立词条，可对照阅读

 **本章定位：**面向零基础/弱基础的 Python 学员，从"为什么需要图数据库"讲起，经过 Docker 部署、Cypher 语法实操，最终落地到用 Python + Neo4j 做 AI 应用的知识图谱和 GraphRAG。前置知识：会用 Python 写基础代码、懂 02 文件的 Docker 基础操作。

目录

前言

- [课程目的与学习意义](#) — 为什么要学图数据库？AI 项目为什么需要图谱？
- [三阶段学习目标](#) — 知道自己要走到哪一步

第一部分：认知层 — 图数据库是什么、为什么

- [为什么需要图数据库](#) — 关系表的痛苦 vs 图的直观
- [Neo4j 是什么](#) — 图数据库的代表
- [图数据库 vs 关系数据库](#) — 一张表看懂本质差异
- [AI 场景速览](#) — GraphRAG / 知识图谱 / Agent 记忆 / 推荐系统

第二部分：实操层 — Docker 部署到熟练使用

- [Docker 部署 Neo4j](#) — 一条命令拉起 + Browser 界面
- [Neo4j Browser](#) — 在浏览器里玩 Cypher
- [Cypher 语法基础](#) — CREATE/MATCH/RETURN
- [节点和关系](#) — 图模型的构建单元
- [关系查询](#) — 多跳查询、路径查找
- [属性过滤和聚合](#) — WHERE/ORDER BY/SKIP/LIMIT/COUNT
- [索引与性能](#) — 让查询更快
- [导入导出数据](#) — CSV 导入、导出 JSON

第三部分：AI 实战层 — Python + Neo4j

- [Python 连接 Neo4j](#) — neo4j 库安装和连接
- [知识图谱构建示例](#) — 从文本提取实体和关系
- [Agent 记忆图谱示例](#) — 带关系的长期记忆
- [GraphRAG 完整项目](#) — 文档→图谱→图增强检索→回答（占 AI 层 60%）
- [其他 AI 场景概览](#) — 推荐系统、语义搜索

工具层

- [Cypher 命令速查表](#) — 按功能分组
- [常见问题](#) — 10+ 个踩坑 FAQ

总结

- [核心概念回顾](#)
- [与后续课程的衔接](#)

课程目的与学习意义

🤖 你可能在想：我是来学 AI 应用开发的，MySQL 和 Redis 已经够用了，为什么要学图数据库？Neo4j 是什么东西？

1. AI 正在从"检索"走向"推理"

传统的 RAG（检索增强生成）只能回答"文档里有没有"的问题。但用户越来越常问的是需要推理的复杂问题：

- "Transformer 和 BERT 是什么关系？"
- "哪些技术属于 Agent 架构？"
- "RLHF 解决的问题和 RAG 有什么不同？"

这些问题需要**知识图谱**——不是查关键词，而是顺着"关系"找到答案。Neo4j 就是做这个的。

2. 有些数据天生就是"图"

社交网络（人认识人）、推荐系统（用户喜欢物品）、知识体系（概念属于分类、概念与概念相关）——这些数据用关系表存需要 5-6 张表 JOIN，用图数据库只需要一行 Cypher 顺着关系走。

3. GraphRAG 是 2024-2025 年最火的 AI 技术趋势之一

微软提出了 GraphRAG（Graph + RAG），用知识图谱增强检索，让 LLM 能回答需要多跳推理的复杂问题。学会了 Neo4j，你就拿到了 GraphRAG 的入场券。

4. 这是 AI 应用开发工程师的高阶技能

打开招聘软件搜"AI 应用开发"或"知识图谱工程师"，你会反复看到：

- "熟悉图数据库（Neo4j 等）"
- "有知识图谱构建经验"
- "了解 GraphRAG 者优先"

5. 为后续 AI 项目铺路

本课程后面的 Agent 开发、知识图谱项目、复杂问答系统——都需要 Neo4j 来做知识建模和图推理。**Neo4j 是 AI 从"检索"升级到"推理"的关键一步。**

三阶段学习目标

阶段	目标	具体表现
一阶段： 能用	Docker 拉起 Neo4j，能打开 Browser，能执行基本 Cypher	<code>docker run</code> 启动、Browser 连接、 <code>CREATE/MATCH/RETURN</code> 不报错

阶段	目标	具体表现
二阶段： 理解	理解图模型和关系表的区别，能写多跳查询，理解关系的方向性	知道什么时候用图数据库，能写 2-3 跳的关系查询
三阶段： 熟练	能用 Python + Neo4j 构建知识图谱，实现 GraphRAG 项目	写出完整的 GraphRAG 系统——文档→图谱→图增强检索→复杂问题回答

💡 **当前这份指南的目标：**帮你从 0 走到三阶段——能部署、能使用 Cypher、能在 AI 项目中实战应用 GraphRAG。

第一部分：认知层 — 图数据库是什么、为什么

在敲命令之前，先搞懂图数据库在干什么。否则你只是在背 Cypher 语法，不理解为什么图模型比关系表更适合某些场景。

为什么需要图数据库

问题场景：你想做一个电影推荐系统，回答"张三可能喜欢什么电影？"

用关系数据库怎么做？

你需要这些表：

用户表 (users):	id, name, age
电影表 (movies):	id, title, year, genre_id
演员表 (actors):	id, name
类型表 (genres):	id, name
评分表 (ratings):	user_id, movie_id, score
出演表 (casts):	movie_id, actor_id

然后写一个查询："找和张三评分相似的用户喜欢的电影，并且这些电影的演员和张三看过的电影有重叠"：

```
-- 这只是简化版，实际更复杂
SELECT DISTINCT m.title
FROM users u1
JOIN ratings r1 ON u1.id = r1.user_id
JOIN movies m ON r1.movie_id = m.id
JOIN casts c ON m.id = c.movie_id
JOIN actors a ON c.actor_id = a.id
WHERE u1.name = '张三'
      AND m.id NOT IN (SELECT movie_id FROM ratings WHERE user_id = u1.id)
-- 还要 JOIN 更多表来找相似用户...
-- 写到第 5 个 JOIN 的时候就已经很难读了
```

😬 **问题很明显：**

- 查询越长越难懂，维护成本高
- 每加一种关系就要加一个 JOIN，性能直线下降
- "多跳查询" (A→B→C→D) 在关系表中要写 3-4 个 JOIN，几乎不可读

用图数据库怎么做？

同样的查询，用图的方式表达：

```
MATCH (张三:User {name: "张三"})-[:LIKES]->(m:Movie)<-[:LIKES]-(other:User)
MATCH (other)-[:LIKES]->(recommend:Movie)
WHERE NOT (张三)-[:LIKES]->(recommend)
RETURN recommend.title
```

 **读起来像在说话：** "找到张三喜欢的电影，其他喜欢这些电影的用户还喜欢的电影，但张三自己还没看过的——返回推荐列表。"

没有 JOIN，没有表关联，就是**顺着关系走**。

直观对比

	关系数据库	图数据库
数据模型	表格，行和列	节点和关系
关系表达	外键 + JOIN	直接连线
多跳查询	多个 JOIN，越长越慢	继续走关系，性能不降
直观性	像填 Excel 表格	像画思维导图
适合场景	结构化数据、统计分析	关系密集型数据、推荐、知识图谱

 **一句话总结：**
如果你的数据核心是"谁和谁有关系"，用图数据库。如果你的数据核心是"记录和统计"，用关系数据库。
两者互补，不是替代关系。

Neo4j 是什么

Neo4j 是最流行的图数据库，也是第一个被广泛使用的原生图数据库。"原生"意味着它从一开始就是为图数据设计的——节点、关系、属性都是第一等公民。

类比理解：
想象你面前有一块白板，上面画着：

- **圆圈（节点）：**代表人、电影、概念、地点.....任何东西\
- **箭头（关系）：**从圆圈指向圆圈，代表"认识"、"喜欢"、"属于"、"位于"\
- **标签（属性）：**贴在圆圈和箭头上小纸条，记录详细信息——名字、年龄、分数\

这就是 Neo4j 的数据模型——白板上的思维导图。你看到的就是实际存储的数据结构，不像关系数据库需要把"关系"翻译成"外键列"和"中间表"。

核心概念

概念	是什么	类比	示例
节点（Node）	图里的"点"——实体对象	思维导图里的圆圈	张三、《黑客帝国》、动作片
关系（Relationship）	点之间的"线"——有方向、有类型	思维导图里的箭头	张三 →喜欢→ 《黑客帝国》
属性（Property）	节点或关系的"详细信息"——键值对	贴在圆圈上的标签纸	<code>{name: "张三", age: 25}</code>
标签（Label）	节点的"分类"——一个节点可以有多个标签	文件柜上的分类标签	<code>:Person</code> 、 <code>:Movie</code> 、 <code>:Genre</code>
路径（Path）	从一个节点走到另一个节点的完整路线	地铁线路图里从 A 站到 B 站的路线	张三 → 喜欢 → 电影 → 演员 → 另一部电影

图数据库 vs 关系数据库

一张表看懂本质差异：

维度	关系数据库（MySQL）	图数据库（Neo4j）
数据模型	表、行、列	节点、关系、属性
关系表达	外键（数字 ID 引用另一行）	直接连接（物理指针）
JOIN 操作	运行时扫描匹配，表越大越慢	直接遍历，和总数据量无关
多跳查询	N 个 JOIN = N 倍性能下降	继续走关系，性能几乎不变
灵活性	改结构要改表（ALTER TABLE）	随时加节点类型和关系，不需要改表
可视化	表格数据，不直观	图结构，天然可视化
典型用途	用户信息、订单、库存、财务	社交网络、推荐系统、知识图谱、欺诈检测

什么时候选谁？

场景	选择	原因
用户注册、订单管理	关系数据库	结构化数据、需要事务和强一致性
缓存、会话、计数器	Redis（内存数据库）	速度快、临时数据
社交关系挖掘	图数据库	朋友的朋友的朋友.....多跳查询
电影/商品推荐	图数据库	用户-物品关系网络

场景	选择	原因
知识图谱、问答系统	图数据库	概念之间的关系推理
文档全文搜索	向量数据库 + BM25	语义匹配

 **AI 项目中的典型组合：**
MySQL（核心数据）+ Redis（缓存）+ Neo4j（知识图谱）+ 向量数据库（语义搜索）
四种数据库各司其职，一个现代 AI 后端通常用到其中 2-3 种。

AI 场景速览

Neo4j 在 AI 领域有几种典型用法，先混个眼熟，后面逐一实战。

1. GraphRAG（图增强检索）★ 本课程重点

问题：传统 RAG 只能按"语义相似度"找文档片段。但用户问"Transformer 和 BERT 有什么关系？"时，关键词不匹配，语义检索可能找不到相关内容。

方案：把知识构建成图谱——术语是节点，关系是"属于"、"与...相关"、"用于"。查询时不仅做语义搜索，还顺着图谱找实体之间的关系，给 LLM 提供更结构化的上下文。

用户提问："Transformer 和 BERT 是什么关系？"

↓

传统 RAG：搜索"Transformer BERT 关系" → 可能找不到直接相关的段落

GraphRAG：在图谱中找到 Transformer 节点 → 沿关系找到 BERT → "BERT 基于 Transformer 的 Encoder-only 架构"

↓

LLM 基于图谱信息生成回答

2. 知识图谱构建

场景：从大量文档、论文、新闻中提取关键实体（人物、公司、技术）和它们之间的关系（合作、竞争、属于），构建一个可查询的知识网络。

AI 应用：让 LLM 能查询"这家公司投资了哪些 AI 初创企业？这些初创企业之间有什么关联？"——不需要重新训练模型，图谱即知识。

3. Agent 记忆图谱

问题：Agent 的普通记忆（List 存最近 N 条对话）是线性的，没有关联。但真实的记忆是网状的——"上次遇到类似问题，我用了 X 工具解决了"。

方案：用图结构存 Agent 的记忆——Agent → 遇到 → 问题 → 使用 → 工具 → 得到 → 结果。这样 Agent 能回答"你上次用什么方法解决了类似问题？"

4. 推荐系统

场景：电商推荐、内容推荐、社交推荐——基于用户-物品关系网络，找到"喜欢 A 的人也喜欢 B"、"关注 X 的人也关注 Y"。

和向量推荐的区别：向量推荐按"相似度"找（意思相近就推荐），图推荐按"关系路径"找（和你行为相似的人喜欢什么就推荐）。两者结合效果更好。

第二部分：实操层 — Docker 部署到熟练使用

认知层搞清楚了。现在进入动手环节——从零开始，一步步学会 Neo4j 的日常操作。

前置检查（花 30 秒确认）：

- ☐ Docker 已安装并能运行（参照 02 文件）
- ☐ Docker Desktop 正在运行
- ☐ 会用 PowerShell/CMD 终端

Docker 部署 Neo4j

类比理解：

装 Neo4j 传统方式需要装 Java、配置环境变量、下载 Neo4j、设密码、启动服务.....一套下来十几分钟。

用 Docker 跑 Neo4j 就像**开一家新店**——docker run 就是"开店"，所有东西都打包好了，直接能用。

一条命令启动

打开 PowerShell 或 CMD：

```
docker run -d --name my-neo4j ^
-p 7474:7474 -p 7687:7687 ^
-e NEO4J_AUTH=neo4j/password123 ^
neo4j:latest
```

这条命令做了什么？ \

- `docker run -d` → 后台运行容器 \
- `--name my-neo4j` → 给容器起名 \
- `-p 7474:7474` → 映射 Browser 界面端口（浏览器访问用） \
- `-p 7687:7687` → 映射 Bolt 协议端口（Python/程序连接用） \
- `-e NEO4J_AUTH=neo4j/password123` → 设置用户名和密码（用户名固定是 neo4j，密码你自定） \
- `neo4j:latest` → 使用最新 Neo4j 镜像

首次下载可能需要 1-2 分钟，Neo4j 镜像比较大（约 500MB）。

验证 Neo4j 是否启动

```
# 查看容器状态
docker ps
```

输出应类似：

CONTAINER ID	IMAGE	PORTS	NAMES
abc123def456	neo4j:latest	0.0.0.0:7474->7474/tcp, 0.0.0.0:7687->7687/tcp	my-neo4j

✅ 看到 **STATUS** 是 **Up** 说明 Neo4j 在跑了。

两个端口都要映射成功：7474（浏览器界面）和 7687（程序连接）。

访问 Neo4j Browser

打开浏览器，访问：

```
http://localhost:7474
```

你会看到 Neo4j 的登录界面：

- 用户名：**neo4j**
- 密码：**password123**（你刚才设置的）

登录后就能看到 Neo4j 的 Browser 界面——一个命令行输入框，下面可以输入 Cypher 查询。

💡 Neo4j Browser 是什么？

它是 Neo4j 自带的可视化工具——你可以在里面输入 Cypher 查询，结果会以**图形方式展示**（节点和关系连线），非常直观。学习 Cypher 的最好方式就是在这里玩。

持久化数据

```
# 停止并删除当前容器
docker stop my-neo4j
docker rm my-neo4j

# 重新启动，挂载数据卷（数据持久化）
docker run -d --name my-neo4j ^
  -p 7474:7474 -p 7687:7687 ^
  -e NEO4J_AUTH=neo4j/password123 ^
  -v C:\neo4j-data\data:/data ^
  -v C:\neo4j-data\logs:/logs ^
  neo4j:latest
```


 **挂载数据卷后**，即使删除容器，Neo4j 的数据文件（数据库、日志）还在 `C:\neo4j-data` 目录下。下次重新启动同一个容器，数据还在。

停止和重启

```
# 停止
docker stop my-neo4j

# 重启（数据还在）
docker start my-neo4j

# 查看日志（排查问题用）
docker logs my-neo4j
```

Neo4j Browser

类比理解：

Neo4j Browser 就像 MySQL 的命令行客户端 + 可视化工具的结合体——你在上面写查询，下面立刻看到图形结果。

打开 Browser

浏览器访问 `http://localhost:7474`，用 `neo4j / password123` 登录。

你会看到：

- 顶部有一个输入框（`:help` 可以查看帮助）
- 下方是结果展示区
- 左侧有数据库信息

第一条 Cypher 查询

在顶部输入框中输入：

```
RETURN "Hello, Neo4j!" AS greeting
```

点击播放按钮（或按 `Ctrl+Enter`），你会看到结果：

```
greeting
"Hello, Neo4j!"
```

 **这说明 Cypher 能正常工作了！**

创建第一个节点和关系

```
// 创建两个节点和一条关系
CREATE (p:Person {name: "张三", age: 25})
CREATE (m:Movie {title: "黑客帝国", year: 1999})
CREATE (p)-[:LIKES {rating: 5}]->(m)
RETURN p, m
```

点击执行后，你会看到**可视化的图结果**：

- 两个圆圈（节点），一个是 Person 标签的"张三"，一个是 Movie 标签的"黑客帝国"
- 中间有一条箭头（关系）从张三指向黑客帝国，标注"LIKES"
- 关系上有个小标签 `rating: 5`

 **这就是 Neo4j 的魅力**——你看到的查询结果就是实际存储的数据结构。不像关系数据库看到的是一个表格，Neo4j 让你"看到"数据的关系。

Browser 常用快捷键

快捷键	功能
<code>Ctrl+Enter</code>	执行查询
<code>Esc</code>	清空输入框
<code>:help</code>	查看帮助
<code>:clear</code>	清除结果区
<code>:sysinfo</code>	查看系统信息

✅ 动手试试：

1. 打开 Neo4j Browser (<http://localhost:7474>)
2. 登录 (neo4j / password123)
3. 执行 `RETURN "Hello, Neo4j!"`
4. 执行上面的 CREATE 查询，看看可视化结果
5. 感受一下——这就是你学习 Cypher 的主战场

Cypher 语法基础

💡 Cypher 是什么？

Cypher 是 Neo4j 的查询语言，专门用来创建、查询、修改、删除图数据。它的设计哲学是***"像画图的 ASCII 艺术"***——用字符画出节点和关系。

基本语法：ASCII 艺术

Cypher 用括号和箭头来表示图结构：

```
(节点)           - 一个节点，像这样： (m:Movie)
(节点)-[关系]->(节点) - 一个带方向的关系： (u)-[:LIKES]->(m)
```

(节点)<-[关系]-(节点) - 反方向的关系: (m)<-[:LIKES]-(u)

📖 记忆技巧:

- `()` 代表节点 (圆圈)
- `[]` 代表关系 (线上的标签)
- `->` 或 `<-` 代表方向 (箭头)
- `:` 后面跟的是类型 (标签名)

CREATE — 创建节点

```
// 创建一个简单节点
CREATE (m:Movie {title: "黑客帝国", year: 1999})

// 一次创建多个节点
CREATE (p:Person {name: "张三"})
CREATE (m:Movie {title: "盗梦空间", year: 2010})
```

📖 语法解释:

- `m` 是这个查询中的变量名 (随便起, 类似 Python 的 `m = ...`)
- `:Movie` 是标签 (Label), 相当于分类
- `{title: "黑客帝国", year: 1999}` 是属性 (Properties), 键值对

MATCH — 查询节点

```
// 查找所有 Movie 类型的节点
MATCH (m:Movie) RETURN m

// 查找特定条件的节点
MATCH (m:Movie {title: "黑客帝国"}) RETURN m

// 只返回特定属性
MATCH (m:Movie) RETURN m.title, m.year
```

📖 **MATCH 和 SQL 的 SELECT 类似**——它不是"创建"而是"找到"已存在的数据。

CREATE + MATCH — 创建关系

```
// 先找到已有的两个节点, 然后创建关系
MATCH (p:Person {name: "张三"})
MATCH (m:Movie {title: "黑客帝国"})
CREATE (p)-[:LIKES {rating: 5, date: "2024-01-01"}]->(m)
```

 **关系也可以有属性**——`{rating: 5}` 就是这条“喜欢”关系的额外信息（打了 5 分）。

完整的创建和查询

```
// 1. 清除之前的数据（学习用，生产环境慎用！）
MATCH (n) DETACH DELETE n

// 2. 创建数据
CREATE (zhangsan:Person {name: "张三", age: 25})
CREATE (lisi:Person {name: "李四", age: 30})
CREATE (movie1:Movie {title: "黑客帝国", year: 1999})
CREATE (movie2:Movie {title: "盗梦空间", year: 2010})
CREATE (movie3:Movie {title: "星际穿越", year: 2014})

// 张三喜欢黑客帝国和盗梦空间
CREATE (zhangsan)-[:LIKES {rating: 5}]->(movie1)
CREATE (zhangsan)-[:LIKES {rating: 4}]->(movie2)

// 李四喜欢盗梦空间和星际穿越
CREATE (lisi)-[:LIKES {rating: 5}]->(movie2)
CREATE (lisi)-[:LIKES {rating: 5}]->(movie3)

// 3. 查询张三喜欢的电影
MATCH (p:Person {name: "张三"})-[:LIKES]->(m:Movie)
RETURN p.name, m.title, m.year
```

输出：

p.name	m.title	m.year
"张三"	"黑客帝国"	1999
"张三"	"盗梦空间"	2010

✅ 动手试试：

1. 在 Browser 中执行上面的完整创建和查询
2. 看看可视化结果——3 个人、3 部电影、4 条关系
3. 试试只查询李四喜欢的电影
4. 修改一个电影属性并重新查询验证

节点和关系

前面学了基本的 CREATE 和 MATCH，现在深入理解图模型的构建单元。

节点详解

```
// 一个节点可以有多个标签
CREATE (p:Person:Actor {name: "基努·里维斯"})

// 属性可以是字符串、数字、布尔值、列表
CREATE (m:Movie {
  title: "黑客帝国",
  year: 1999,
  rating: 8.7,
  active: true,
  genres: ["科幻", "动作"]
})
```

关系详解

```
// 关系必须有类型（如 :LIKES, :ACTED_IN, :DIRECTED）
MATCH (p:Person {name: "基努·里维斯"})
MATCH (m:Movie {title: "黑客帝国"})
CREATE (p)-[:ACTED_IN {role: "Neo"}]->(m)

// 关系是有方向的
// ACTED_IN 的方向：演员 → 出演 → 电影
// 查询时要顺着方向走：
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name, m.title

// 如果要反方向查，用 <-
MATCH (m:Movie)<-[:ACTED_IN]-(p:Person)
RETURN m.title, p.name
```

关系方向的重要性：

ACTED_IN 的语义是“演员出演电影”，所以方向从 Person 到 Movie。

如果你想查“这部电影有哪些演员”，你可以反向查 **<-[:ACTED_IN]-**。

Cypher 中，方向不影响查询结果——(a)-[:R]->(b)** 和 **(a)<-[:R]-(b)** 是一样的，但方向影响语义理解。**

变量复用

```
// 在同一个 CREATE 中复用变量，一次性创建节点和关系
CREATE (p:Person {name: "王五"})
  -[:LIKES {rating: 4}]->
  (m:Movie {title: "阿甘正传", year: 1994})
RETURN p, m
```

 这行创建了 1 个 Person 节点、1 个 Movie 节点、1 条关系——一行搞定。

唯一性约束

```
// 创建唯一约束（同名同年的电影只存一份）
CREATE CONSTRAINT movie_title_year FOR (m:Movie)
  REQUIRE (m.title, m.year) IS UNIQUE

// 之后 CREATE 重复的会报错
```

💡 学习阶段可以不设约束，但生产环境一定要设——防止数据重复。

常用关系类型示例

关系类型	方向	含义	示例
:LIKES	Person → Movie	用户喜欢电影	张三 → 喜欢 → 黑客帝国
:ACTED_IN	Person → Movie	演员出演电影	基努·里维斯 → 出演 → 黑客帝国
:DIRECTED	Person → Movie	导演执导	沃卓斯基姐妹 → 执导 → 黑客帝国
:BELONGS_TO	Movie → Genre	电影属于类型	黑客帝国 → 属于 → 科幻
:SIMILAR_TO	Movie → Movie	电影相似	黑客帝国 → 相似 → 盗梦空间
:KNOWS	Person → Person	人认识人	张三 → 认识 → 李四

✅ 动手试试：

1. 给之前的图谱添加演员和导演节点

2. 用 :ACTED_IN 和 :DIRECTED 关系连接

3. 查询"黑客帝国的演员有哪些"

4. 查询"张三和李四共同喜欢的电影"

关系查询

图数据库的杀手级功能——**多跳查询**。在关系表中需要多个 JOIN 的操作，在 Neo4j 中只是"继续沿着关系走"。

单跳查询（已经学过）

```
// 张三喜欢的电影
MATCH (p:Person {name: "张三"})-[:LIKES]->(m:Movie)
RETURN m.title
```

两跳查询（朋友推荐）

```
// 找到和张三喜欢相同电影的人（朋友），再看他们喜欢什么
MATCH (zhangsan:Person {name: "张三"})-[:LIKES]->(m:Movie)<-[:LIKES]-
```

```
(friend:Person)
MATCH (friend)-[:LIKES]->(recommend:Movie)
WHERE recommend <> m // 排除已经喜欢的
RETURN friend.name, collect(DISTINCT recommend.title) AS recommendations
```

读这段 Cypher:

1. 张三 → 喜欢 → 电影 m ← 喜欢 ← 朋友（找到和张三品味相似的人）
2. 朋友 → 喜欢 → 推荐电影
3. 过滤掉张三已经看过的
4. 返回朋友名字和他们推荐的不重复电影列表

三跳查询（演员推荐）

```
// 张三喜欢的电影的演员，还演了什么其他电影？
MATCH (p:Person {name: "张三"})-[:LIKES]->(m:Movie)<-[:ACTED_IN]-(actor:Person)
MATCH (actor)-[:ACTED_IN]->(other:Movie)
WHERE other <> m
RETURN DISTINCT other.title, actor.name
```

可变深度路径

```
// 找到和张三"认识"链条上的所有人（1-3 度人脉）
MATCH path = (zhangsan:Person {name: "张三"})-[:KNOWS*1..3]-(other:Person)
RETURN other.name, length(path) AS degrees
ORDER BY degrees
```

*1..3 是什么？

可变长度关系——走 1 到 3 步的 **:KNOWS** 关系。 \

- *1 = 直接认识（朋友） \
- *2 = 朋友的朋友（二度人脉） \
- *3 = 朋友的朋友的朋友（三度人脉） \

这在关系表中几乎不可能高效查询（需要递归 CTE），在 Neo4j 中一行搞定。

最短路径

```
// 张三和李四之间最短的认识路径
MATCH path = shortestPath(
  (zhangsan:Person {name: "张三"})-[:KNOWS*]-(lisi:Person {name: "李四"})
)
RETURN path
```

关系查询速查

模式	说明	示例
<code>(a)-[:R]->(b)</code>	单跳查询	用户喜欢的电影
<code>(a)-[:R]->(b)-[:R2]->(c)</code>	两跳查询	朋友推荐
<code>(a)-[:R*1..n]-(b)</code>	可变深度	N 度人脉
<code>shortestPath((a)-[*]-(b))</code>	最短路径	两人怎么认识的
<code>(a)-[:R*]->(b)</code>	任意深度	所有可达节点

- ✔ 动手试试：
1. 给图谱添加几条 `:KNOWS` 关系（张三认识李四，李四认识王五...）

2. 用 `*1..3` 查询张三的 3 度人脉

3. 尝试找到两个节点之间的最短路径

属性过滤和聚合

和 SQL 类似，Cypher 也支持 WHERE 过滤、ORDER BY 排序、聚合函数等。

WHERE 过滤

```
// 查找 2000 年以后的电影
MATCH (m:Movie)
WHERE m.year > 2000
RETURN m.title, m.year

// 多条件过滤
MATCH (m:Movie)
WHERE m.year >= 2000 AND m.year <= 2020
RETURN m.title, m.year

// 模糊匹配
MATCH (p:Person)
WHERE p.name CONTAINS "张"
RETURN p.name

// 列表包含
MATCH (m:Movie)
WHERE "科幻" IN m.genres
RETURN m.title
```

ORDER BY


```
// 按评分排序
MATCH (p:Person)-[r:LIKES]->(m:Movie)
WHERE p.name = "张三"
RETURN m.title, r.rating
ORDER BY r.rating DESC

// 多字段排序
MATCH (m:Movie)
RETURN m.title, m.year
ORDER BY m.year DESC, m.title ASC
```

SKIP / LIMIT

```
// 查看第 2 页（每页 10 条）
MATCH (m:Movie)
RETURN m.title
ORDER BY m.year DESC
SKIP 10 LIMIT 10
```

聚合函数

```
// 计数
MATCH (m:Movie)
RETURN count(m) AS totalMovies

// 按用户分组，计算每人喜欢的电影数
MATCH (p:Person)-[:LIKES]->(m:Movie)
RETURN p.name, count(m) AS movieCount
ORDER BY movieCount DESC

// 平均分
MATCH (p:Person)-[r:LIKES]->(m:Movie {title: "黑客帝国"})
RETURN avg(r.rating) AS averageRating

// 收集为列表
MATCH (p:Person {name: "张三"})-[:LIKES]->(m:Movie)
RETURN p.name, collect(m.title) AS likedMovies

// 最大值/最小值
MATCH (m:Movie)
RETURN max(m.year) AS newest, min(m.year) AS oldest
```

聚合速查

函数	说明	示例
count(n)	计数	count(m) AS total
avg(prop)	平均值	avg(r.rating)
sum(prop)	求和	sum(r.rating)
max(prop)	最大值	max(m.year)
min(prop)	最小值	min(m.year)
collect(prop)	收集为列表	collect(m.title)
DISTINCT	去重	collect(DISTINCT m.title)

✔ 动手试试：

1. 查询每个电影的平均评分

2. 找出评分最高的电影

3. 按年份统计每年有多少电影

4. 找张三最喜欢的电影类型（count genres）

索引与性能

⚡ 为什么需要索引？

当你的图谱有几万个节点时，每次 MATCH (m:Movie {title: "黑客帝国"}) 都要遍历所有 Movie 节点来找匹配的那个。索引就像书的目录——直接翻到那一页，不用一页一页翻。

创建索引

```
// 为 Movie.title 创建索引
CREATE INDEX movieTitleIndex FOR (m:Movie) ON (m.title)

// 为多个属性创建复合索引
CREATE INDEX movieYearGenre FOR (m:Movie) ON (m.year, m.genres)

// 查看已创建的索引
SHOW INDEXES
```

查看查询计划

```
// 看看查询是否用到了索引
EXPLAIN MATCH (m:Movie {title: "黑客帝国"}) RETURN m
```

执行后，Browser 会展示查询计划（Query Plan）——一个可视化的执行流程图。如果你看到 **NodeIndexSeek**，说明用到了索引；如果看到 **NodeByLabelScan**，说明在扫描全表（慢）。

性能调优建议

建议	说明
给常用查询条件加索引	特别是 WHERE 过滤的字段
避免全表扫描	<code>MATCH (n)</code> 返回所有节点在大数据量时很慢
用 <code>EXPLAIN</code> 分析查询	定期查看查询计划，优化慢查询
限制返回数量	加 <code>LIMIT</code> 避免一次性返回太多数据
关系比属性查询快	频繁查询的关系建议建模为关系而不是属性

导入导出数据

实际项目中，你的数据通常在 CSV、JSON 文件中，而不是手动一条条 CREATE。

从 CSV 导入

假设你有一个 `movies.csv`：

```
title,year,genre
黑客帝国,1999,科幻
盗梦空间,2010,科幻
星际穿越,2014,科幻
```

在 Neo4j Browser 中：

```
// 从文件导入（文件需要放在 Neo4j 的 import 目录）
LOAD CSV WITH HEADERS FROM 'file:///movies.csv' AS row
CREATE (m:Movie {
  title: row.title,
  year: toInteger(row.year),
  genre: row.genre
})
```

 **Docker 环境中**，需要把 CSV 文件放到容器的 `/var/lib/neo4j/import` 目录：

```
# 把本地 CSV 复制到容器的 import 目录
docker cp C:\data\movies.csv my-neo4j:/var/lib/neo4j/import/
```

导入关系

假设有一个 `ratings.csv`：

```
userName,movieTitle,rating
张三,黑客帝国,5
张三,盗梦空间,4
李四,盗梦空间,5
```

```
LOAD CSV WITH HEADERS FROM 'file:///ratings.csv' AS row
MATCH (p:Person {name: row.userName})
MATCH (m:Movie {title: row.movieTitle})
CREATE (p)-[:LIKES {rating: toInteger(row.rating)}]->(m)
```

为什么先 MATCH 再 CREATE?

因为用户和电影节点已经存在了，先找到它们，再创建关系。

导出数据

```
// 查询结果导出为 JSON（在 Python 中更容易）
MATCH (m:Movie)
RETURN m.title, m.year, m.genre
```

然后在 Python 中获取结果并保存为 JSON 文件。

第三部分：AI 实战层 — Python + Neo4j

认知和实操都搞定了。现在进入最重要的部分——**用 Python 连接 Neo4j，在 AI 项目中真正用起来。**

Python 连接 Neo4j

安装 neo4j 库

```
pip install neo4j
```

 **neo4j** 是 Neo4j 官方的 Python 驱动，支持 Bolt 协议（Neo4j 的高性能网络协议）。

测试连接

```
from neo4j import GraphDatabase

# 连接到 Neo4j
uri = "bolt://localhost:7687"
driver = GraphDatabase.driver(uri, auth=("neo4j", "password123"))
```

```
# 测试连接
with driver.session() as session:
    result = session.run("RETURN 'Hello, Neo4j!' AS greeting")
    record = result.single()
    print(record["greeting"]) # Hello, Neo4j!

# 关闭连接（程序退出时）
driver.close()
```

 **重要：**uri 用的是 `bolt://` 而不是 `http://`。Bolt 是 Neo4j 的高效二进制协议，端口是 7687（不是 Browser 的 7474）。

封装为工具类

```
from neo4j import GraphDatabase

class Neo4jHelper:
    """Neo4j 数据库辅助类"""

    def __init__(self, uri="bolt://localhost:7687", user="neo4j",
password="password123"):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))

    def close(self):
        self.driver.close()

    def run_query(self, query, params=None):
        """执行查询，返回结果列表"""
        with self.driver.session() as session:
            result = session.run(query, params or {})
            return [record.data() for record in result]

    def run_write(self, query, params=None):
        """执行写入操作"""
        with self.driver.session() as session:
            session.run(query, params or {})

# 测试
helper = Neo4jHelper()
print(helper.run_query("RETURN 'Hello!' AS msg"))
helper.close()
```

✅ 动手试试：

1. `pip install neo4j`
2. 确保 Docker 里的 Neo4j 在运行
3. 运行上面的测试代码
4. 确认能连接、能查询

知识图谱构建示例

目标：从 01 文件的 AI 术语手册中提取术语和关系，构建一个简单的知识图谱。

```
from neo4j import GraphDatabase

class KnowledgeGraphBuilder:
    """知识图谱构建器 - 从术语数据构建 AI 知识图谱"""

    def __init__(self, uri="bolt://localhost:7687", user="neo4j",
password="password123"):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))

    def close(self):
        self.driver.close()

    def add_concept(self, name: str, category: str, description: str):
        """添加一个概念节点"""
        query = """
MERGE (c:Concept {name: $name})
SET c.category = $category, c.description = $description
"""
        with self.driver.session() as session:
            session.run(query, name=name, category=category,
description=description)

    def add_relation(self, from_concept: str, relation_type: str, to_concept:
str):
        """添加概念之间的关系"""
        query = """
MATCH (a:Concept {name: $from})
MATCH (b:Concept {name: $to})
MERGE (a)-[r:RELATED {type: $rel_type}]->(b)
"""
        with self.driver.session() as session:
            session.run(query, from_=from_concept, to=to_concept,
rel_type=relation_type)

    def get_related_concepts(self, concept_name: str) -> list:
        """查找和某个概念相关的所有概念"""
        query = """
MATCH (c:Concept {name: $name})-[r:RELATED]->(other:Concept)
RETURN other.name AS name, r.type AS relation
"""
        with self.driver.session() as session:
            result = session.run(query, name=concept_name)
            return [record.data() for record in result]

    def get_concepts_by_category(self, category: str) -> list:
        """查找某个分类下的所有概念"""
        query = """
MATCH (c:Concept {category: $category})
"""
```

```

        RETURN c.name AS name, c.description AS description
        """
        with self.driver.session() as session:
            result = session.run(query, category=category)
            return [record.data() for record in result]

# === 构建知识图谱 ===
if __name__ == "__main__":
    builder = KnowledgeGraphBuilder()

    print("📊 构建 AI 知识图谱...\n")

    # 添加概念 (从 01 文件提取)
    concepts = [
        ("Transformer", "模型架构", "一眼看全局、自动划重点的架构"),
        ("BERT", "模型架构", "前后都看, 像做完形填空"),
        ("GPT", "模型架构", "逐字生成的 Decoder-only 模型"),
        ("RAG", "知识增强", "AI 先查资料再回答, 开卷考试"),
        ("Fine-tuning", "知识增强", "用专业知识再培训, 通才变专家"),
        ("RLHF", "模型优化", "人类给 AI 打分, 教它好好说话"),
        ("Agent", "Agent架构", "能自己想办法干活的 AI"),
        ("MCP", "Agent架构", "AI 的万能 USB-C 接口"),
        ("Prompt Engineering", "提示交互", "怎么提问, AI 才给你好答案"),
    ]

    for name, category, desc in concepts:
        builder.add_concept(name, category, desc)
        print(f"✅ 添加概念: {name} ({category}) ")

    print("\n🔗 添加关系...\n")

    # 添加关系
    relations = [
        ("BERT", "基于", "Transformer"),
        ("GPT", "基于", "Transformer"),
        ("RAG", "增强", "Fine-tuning"),
        ("RLHF", "优化", "GPT"),
        ("Agent", "使用", "Prompt Engineering"),
        ("Agent", "使用", "MCP"),
        ("RAG", "需要", "Prompt Engineering"),
    ]

    for from_c, rel_type, to_c in relations:
        builder.add_relation(from_c, rel_type, to_c)
        print(f"🔗 {from_c} --[{rel_type}]-> {to_c}")

    # 查询验证
    print("\n🔍 Transformer 相关的技术:")
    related = builder.get_related_concepts("Transformer")
    for r in related:
        print(f"    • {r['name']} (关系: {r['relation']}) ")

    print("\n📁 Agent 架构下的技术:")

```

```
agents = builder.get_concepts_by_category("Agent架构")
for a in agents:
    print(f"    {a['name']}: {a['description']}")

builder.close()
print("\n✅ 知识图谱构建完成！")
```

📖 关键技巧：

- **MERGE** = "存在就不创建，不存在就创建"——防止重复节点\
- 用分类（category）属性做快速过滤\
- 关系类型（type）作为关系的属性，灵活描述不同的语义

Agent 记忆图谱示例

问题：Agent 的普通记忆是线性的（List 存最近 N 条对话），但真实的记忆是网状的——事件之间有关联、经验可以复用。

方案：用图结构存 Agent 的"经历"——Agent 遇到什么问题、用了什么工具、得到了什么结果、学到了什么经验。

0

📖 为什么用图存记忆？

- 不是简单的"对话列表"，而是**事件网络**——Agent 能回答"我以前用什么工具解决了类似问题？"
- 关系链：Agent → 遇到 → 问题 → 使用 → 工具 → 得到 → 结果 → 教会 → 经验\
- 新遇到问题时，顺着图谱找到之前用过的工具和教训

GraphRAG 完整项目

🎯 **这是本课程的压轴项目**——把知识图谱和 LLM 结合，构建一个能回答复杂问题的 GraphRAG 系统。

项目目标

传统 RAG 的局限：

- 只能按"语义相似度"检索文档
- 回答不了需要"关系推理"的问题（如"Transformer 和 BERT 的关系"）

GraphRAG 的方案：

- 先把知识构建为图谱（术语、关系、分类）
- 查询时**先查图谱找到相关实体和关系**，再用语义检索文档
- 给 LLM 提供**结构化上下文**（图谱关系 + 文档片段），回答更准确

项目架构

用户提问 → 实体提取 → 图谱查询 → 构建关系上下文 → LLM 回答
 ↓
 语义检索（可选补充）

完整代码

```
"""
GraphRAG - 图增强检索系统
结合 Neo4j 知识图谱 + LLM 回答复杂问题
"""

from neo4j import GraphDatabase
import json

class GraphRAGSystem:
    """GraphRAG 系统 - 用知识图谱增强 LLM 的检索和推理能力"""

    def __init__(self, neo4j_uri="bolt://localhost:7687",
                 neo4j_user="neo4j", neo4j_password="neo4j123"):
        self.driver = GraphDatabase.driver(
            neo4j_uri, auth=(neo4j_user, neo4j_password))
        self._init_graph()

    def close(self):
        self.driver.close()

    def _init_graph(self):
        """初始化知识图谱（如果没有 Concept 数据或关系的话）"""
        with self.driver.session() as session:
            concept_count = session.run("MATCH (c:Concept) RETURN count(c) AS cnt").single()
            rel_count = session.run("MATCH ()-[r:RELATED]->() RETURN count(r) AS cnt").single()
            if concept_count["cnt"] > 0 and rel_count["cnt"] > 0:
                print(f"📊 知识图谱已存在 ({concept_count['cnt']} 个概念, {rel_count['cnt']} 条关系), 跳过初始化\n")
                return # 已有完整数据, 跳过初始化

            print("📊 初始化知识图谱...")

            # 批量导入概念
            concepts = [
                {"name": "Transformer", "category": "模型架构",
                 "description": "Attention Is All You Need 提出的架构, 一眼看全局、自动划重点"},
                {"name": "BERT", "category": "模型架构",
                 "description": "Google 的 Encoder-only 模型, 前后都看, 像做完形填空"},
                {"name": "GPT", "category": "模型架构",
```

```

    "description": "OpenAI 的 Decoder-only 模型, 逐字生成"},
{"name": "LLaMA", "category": "模型架构",
 "description": "Meta 的开源大模型, 任何人都能下载"},
{"name": "RAG", "category": "知识增强",
 "description": "检索增强生成—AI 先查资料再回答, 开卷考试"},
{"name": "Fine-tuning", "category": "知识增强",
 "description": "微调—用专业数据再训练, 通才变专家"},
{"name": "RLHF", "category": "模型优化",
 "description": "人类反馈强化学习—人类给回答打分, 教 AI 好好说话"},
{"name": "Agent", "category": "Agent架构",
 "description": "能自主感知环境、规划行动并完成复杂任务的 AI"},
{"name": "MCP", "category": "Agent架构",
 "description": "Model Context Protocol—AI 的万能 USB-C 接口"},
{"name": "Function Calling", "category": "Agent架构",
 "description": "让 AI 调用外部工具, 从聊天变成能干活"},
{"name": "Prompt Engineering", "category": "提示交互",
 "description": "提示词工程—怎么提问, AI 才给你好答案"},
{"name": "Chain of Thought", "category": "推理范式",
 "description": "思维链—让 AI 一步步写出推理过程"},
{"name": "Embedding", "category": "知识增强",
 "description": "把文字变成数字向量, 语义相近的数字相近"},
{"name": "Vector Database", "category": "知识增强",
 "description": "向量数据库—按语义相似度找东西"},
{"name": "Token", "category": "模型基础",
 "description": "AI 读写的最小单位, 像积木块"},
{"name": "Hallucination", "category": "模型基础",
 "description": "幻觉—AI 一本正经地胡说八道"},
{"name": "Context Window", "category": "模型基础",
 "description": "上下文窗口—AI 一次能记住的最大文字量"},
]

for c in concepts:
    session.run("""
CREATE (c:Concept {
    name: $name,
    category: $category,
    description: $description
})
""", **c)

# 批量导入关系
relations = [
    ("BERT", "基于", "Transformer"),
    ("GPT", "基于", "Transformer"),
    ("LLaMA", "基于", "Transformer"),
    ("RAG", "补充", "Fine-tuning"),
    ("RAG", "使用", "Embedding"),
    ("RAG", "使用", "Vector Database"),
    ("RLHF", "优化", "GPT"),
    ("Agent", "使用", "Prompt Engineering"),
    ("Agent", "使用", "Function Calling"),
    ("Agent", "使用", "MCP"),
    ("Chain of Thought", "提升", "Prompt Engineering"),
    ("Hallucination", "被缓解", "RAG"),

```

```

        ("Context Window", "限制", "Token"),
        ("Embedding", "用于", "Vector Database"),
    ]

    for from_name, rel_type, to_name in relations:
        session.run("""
            MATCH (a:Concept {name: $from_name})
            MATCH (b:Concept {name: $to_name})
            MERGE (a)-[r:RELATED {type: $rel_type}]->(b)
            """, from_name=from_name, to_name=to_name, rel_type=rel_type)

    print(f"✅ 知识图谱初始化完成 ({len(concepts)} 个概念, {len(relations)} 条关系) \n")

def query_graph_context(self, question: str) -> str:
    """
    从问题中提取关键术语，查询图谱找到相关概念和关系，
    返回结构化的图谱上下文。
    """
    # 简单实现：在问题中搜索已知概念名
    # 生产环境这里会用 NER（命名实体识别）来提取实体
    concepts_in_question = []

    with self.driver.session() as session:
        all_concepts = session.run("MATCH (c:Concept) RETURN c.name AS name")
        for record in all_concepts:
            name = record["name"]
            # 检查概念名是否出现在问题中（忽略大小写）
            if name.lower() in question.lower():
                concepts_in_question.append(name)

    if not concepts_in_question:
        return "（未找到图谱中的相关概念）"

    # 查询这些概念的关系
    context_parts = []
    for concept in concepts_in_question:
        with self.driver.session() as session:
            # 查找直接关系
            result = session.run("""
                MATCH (c:Concept {name: $name})-[r:RELATED]->(other:Concept)
                RETURN other.name AS related, r.type AS relation_type
                """, name=concept)

            relations = [f" - {record['related']}"
                        (f"{record['relation_type']})"
                        for record in result]

            if relations:
                context_parts.append(f"💡 {concept} 的关系: \n" +
                                     "\n".join(relations))

    # 查找同分类的概念
    result = session.run("""

```

```

MATCH (c:Concept {name: $name})
MATCH (other:Concept {category: c.category})
WHERE other.name <> c.name
RETURN other.name AS name
""", name=concept)

siblings = [record["name"] for record in result]
if siblings:
    cat_result = session.run("""
MATCH (c:Concept {name: $name}) RETURN c.category AS cat
""", name=concept)
    cat = cat_result.single()["cat"]
    context_parts.append(f"📁 {concept} 的同分类 ({cat}) 概念: {'',
'.join(siblings)}")

return "\n\n".join(context_parts)

def get_concept_description(self, concept_name: str) -> str:
    """获取某个概念的描述"""
    with self.driver.session() as session:
        result = session.run("""
MATCH (c:Concept {name: $name})
RETURN c.description AS desc
""", name=concept_name)
        record = result.single()
        return record["desc"] if record else ""

def answer_question(self, question: str) -> str:
    """
    GraphRAG 完整流程：
    1. 从问题提取实体
    2. 查图谱找关系
    3. 构建增强上下文
    4. 构造 prompt 给 LLM
    5. 返回回答
    """
    print(f" ? 用户问题: {question}\n")

    # 第 1-2 步：查询图谱上下文
    graph_context = self.query_graph_context(question)
    print(f"📁 图谱上下文: \n{graph_context}\n")

    # 第 3 步：构建增强 prompt
    prompt = f"""你是一个 AI 技术专家，请根据以下知识图谱信息和你的专业知识回答用户
的问题。

知识图谱中的相关信息：
{graph_context}

用户问题: {question}

请基于图谱信息给出准确、通俗的回答。如果图谱信息不足以完全回答问题，请补充你的知识。"""

    # 第 4 步：模拟 LLM 回答（实际项目中替换为 API 调用）

```

```

print("🤖 生成回答...")
response = self._mock_llm(prompt, question)

return response

def _mock_llm(self, prompt: str, question: str) -> str:
    """模拟 LLM 回答（实际项目中替换为 openai/anthropic API 调用）"""
    # 这里根据问题关键词给出模拟回答
    answers = {
        "Transformer": "Transformer 是 2017 年论文《Attention Is All You Need》提出的架构，核心是自注意力机制（Self-Attention）——一眼看完整个句子，自动找出哪些词之间有联系。BERT、GPT、LLaMa 等所有现代大模型都基于 Transformer。",
        "BERT": "BERT 是 Google 2018 年发布的 Encoder-only 模型，训练时遮住一些词让模型猜（完形填空），所以它能同时看左右文（双向理解）。基于 Transformer 的 Encoder 部分。",
        "GPT": "GPT 是 OpenAI 的 Decoder-only 模型，从左到右逐字生成文字。基于 Transformer 的 Decoder 部分。ChatGPT 就是基于 GPT 架构。",
        "RAG": "RAG（检索增强生成）让 AI 在回答前先查知识库，相当于开卷考试。它使用 Embedding 把文字变成向量，存在 Vector Database 里按语义相似度检索。能有效缓解 Hallucination（幻觉）。",
        "Agent": "Agent（AI 智能体）是能自主感知、规划、行动的 AI，不只是聊天。它使用 Prompt Engineering 来引导行为、Function Calling 来调用外部工具、MCP 来连接各种数据源。",
        "Fine-tuning": "Fine-tuning（微调）是在通用大模型基础上用专业数据继续训练，让通才变专家。和 RAG 不同，Fine-tuning 是把知识内化到模型参数里，而 RAG 是外挂知识库。",
        "RLHF": "RLHF（人类反馈强化学习）让人类给 AI 的回答打分，教它什么叫‘说得好’——不是说得更准确，而是说得更符合人类偏好。ChatGPT 的对话能力就大量依赖 RLHF。",
    }

    for keyword, answer in answers.items():
        if keyword.lower() in question.lower():
            return answer

    return f"关于这个问题，图谱中暂时没有直接匹配的信息。根据图谱的关系信息，我可以告诉你：\n\n{prompt}\n\n（在实际项目中，这里会调用真实的 LLM API 来生成回答。）"

# === 运行项目 ===
if __name__ == "__main__":
    rag = GraphRAGSystem()

    print("=" * 70)
    print("🧠 GraphRAG – 图增强检索系统")
    print("=" * 70)

    # 测试问题 1：简单概念查询
    print("\n" + "=" * 70)
    answer = rag.answer_question("BERT 是什么？它和 Transformer 有什么关系？")
    print(f"\n💡 回答：{answer}\n")

    # 测试问题 2：关系推理
    print("=" * 70)
    answer = rag.answer_question("GPT 和 BERT 有什么区别？它们的底层架构是什么？")

```

```

print(f"\n💡 回答: {answer}\n")

# 测试问题 3: 场景问题
print("=" * 70)
answer = rag.answer_question("Agent 需要用到哪些技术? ")
print(f"\n💡 回答: {answer}\n")

# 测试问题 4: 对比问题
print("=" * 70)
answer = rag.answer_question("RAG 和 Fine-tuning 有什么区别? ")
print(f"\n💡 回答: {answer}\n")

rag.close()
print("=" * 70)
print("✅ GraphRAG 项目演示完成")
print("=" * 70)

```

运行输出示例:

```

=====
💡 GraphRAG - 图增强检索系统
=====

=====
? 用户问题: BERT 是什么? 它和 Transformer 有什么关系?

图谱上下文:
✦ BERT 的关系:
- Transformer (基于)

📁 BERT 的同分类 (模型架构) 概念: Transformer, GPT, LLaMA

🖨️ 生成回答...

💡 回答: BERT 是 Google 2018 年发布的 Encoder-only 模型, 训练时遮住一些词让模型猜 (完形填空), 所以它能同时看左右文 (双向理解)。基于 Transformer 的 Encoder 部分。

=====
? 用户问题: GPT 和 BERT 有什么区别? 它们的底层架构是什么?

图谱上下文:
✦ GPT 的关系:
- Transformer (基于)
- RLHF (被优化)

✦ BERT 的关系:
- Transformer (基于)

📁 GPT 的同分类 (模型架构) 概念: Transformer, BERT, LLaMA

🖨️ 生成回答...

```

💡 回答：GPT 是 OpenAI 的 Decoder-only 模型，从左到右逐字生成文字。基于 Transformer 的 Decoder 部分。BERT 是 Encoder-only 模型，能同时看左右文（双向理解）。基于 Transformer 的 Encoder 部分。两者都基于 Transformer，但使用的部分不同。

...（后续问题类似）

📖 GraphRAG 的核心价值：\

1. **图谱提供结构化关系**——不只是"语义相似"，而是"基于"、"使用"、"属于"等精确语义关系\
2. **LLM 有了准确的事实依据**——不是纯靠训练数据猜，而是有图谱中的权威关系\
3. **能回答多跳推理问题**——"BERT 和 Transformer 的关系"需要顺关系链（BERT → 基于 → Transformer），传统 RAG 很难做到\

实际项目中，把 `_mock_llm` 方法替换为真实的 `openai` 或 `anthropic` API 调用，这就是一个完整的 GraphRAG 系统。

其他 AI 场景概览

推荐系统：基于关系的推荐

```
# 找到"喜欢 A 电影的人也喜欢的电影"
query = """
MATCH (user:User {name: $name})-[:LIKES]->(m:Movie)<-[:LIKES]-(other:User)
MATCH (other)-[:LIKES]->(recommend:Movie)
WHERE NOT (user)-[:LIKES]->(recommend)
RETURN recommend.title, count(*) AS recommendCount
ORDER BY recommendCount DESC
LIMIT 10
"""
```

📖 **和向量推荐的区别**：向量推荐按"内容相似度"找（这部电影和你喜欢的那部意思相近），图推荐按"人群行为"找（喜欢相同电影的人还看了什么）。两者结合就是"内容+协同"的混合推荐。

语义搜索：图谱+向量双引擎

```
# 图谱查询：找到相关概念
graph_results = helper.run_query("""
    MATCH (c:Concept)
    WHERE c.name CONTAINS $keyword OR c.description CONTAINS $keyword
    RETURN c.name, c.description, c.category
""", params={"keyword": "注意力"})

# 向量检索：找到语义相似的文档
# （这里需要向量数据库，如 Milvus / Chroma）
# 两者结果合并后给 LLM
```

工具层

日常操作你已经会了，AI 场景也实战过了。下面是一些让 Neo4j 更好用的细节。

Cypher 命令速查表

按功能分组的日常最常用命令：

创建

命令	说明	示例
CREATE (n:Label)	创建节点	CREATE (m:Movie {title: "黑客帝国"})
CREATE (a)-[:R]->(b)	创建关系	CREATE (p)-[:LIKES]->(m)
MERGE (n:Label)	存在就不创建	MERGE (c:Concept {name: "BERT"})

查询

命令	说明	示例
MATCH (n:Label)	查找节点	MATCH (m:Movie) RETURN m
MATCH (a)-[:R]->(b)	顺关系查	MATCH (u)-[:LIKES]->(m) RETURN m
MATCH (a)-[:R*1..n]-(b)	可变深度	MATCH path = (a)-[:KNOWS*1..3]-(b) RETURN path
shortestPath((a)-[*]-(b))	最短路径	MATCH p = shortestPath((a)-[*]-(b)) RETURN p

过滤和聚合

命令	说明	示例
WHERE	条件过滤	WHERE m.year > 2000
ORDER BY	排序	ORDER BY m.year DESC
SKIP n LIMIT m	分页	SKIP 10 LIMIT 10
count()	计数	count(m) AS total
collect()	收集为列表	collect(m.title)
avg() / sum() / max() / min()	聚合	avg(r.rating)

修改和删除

命令	说明	示例
SET n.prop = val	设置属性	SET m.year = 2000
REMOVE n.prop	删除属性	REMOVE m.old_field

命令	说明	示例
DELETE n	删除节点	MATCH (m:Movie) DELETE m
DETACH DELETE n	删除节点和关系	MATCH (n) DETACH DELETE n

索引

命令	说明	示例
CREATE INDEX	创建索引	CREATE INDEX movieTitle FOR (m:Movie) ON (m.title)
SHOW INDEXES	查看索引	SHOW INDEXES
EXPLAIN	查看执行计划	EXPLAIN MATCH (m:Movie {title: "xx"}) RETURN m

导入

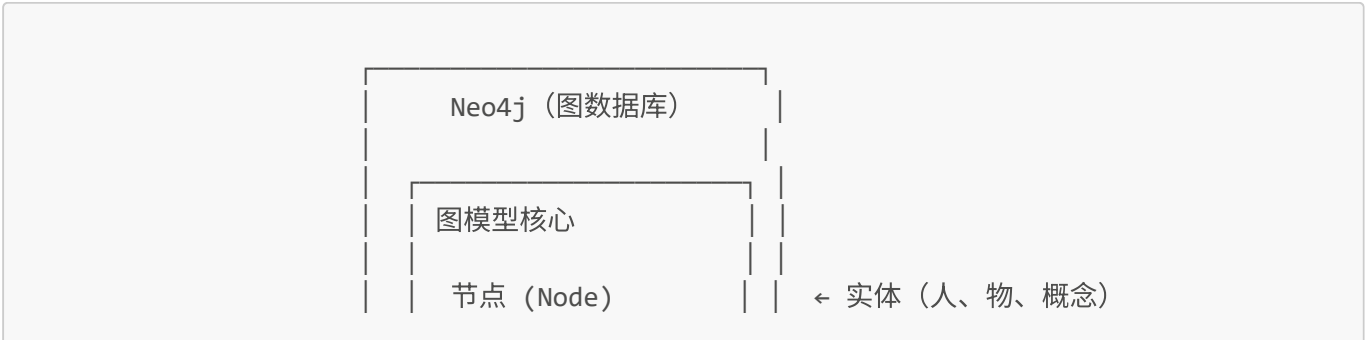
命令	说明	示例
LOAD CSV	导入 CSV	LOAD CSV WITH HEADERS FROM 'file:///data.csv' AS row
MERGE	去重导入	MERGE (c:Concept {name: row.name})

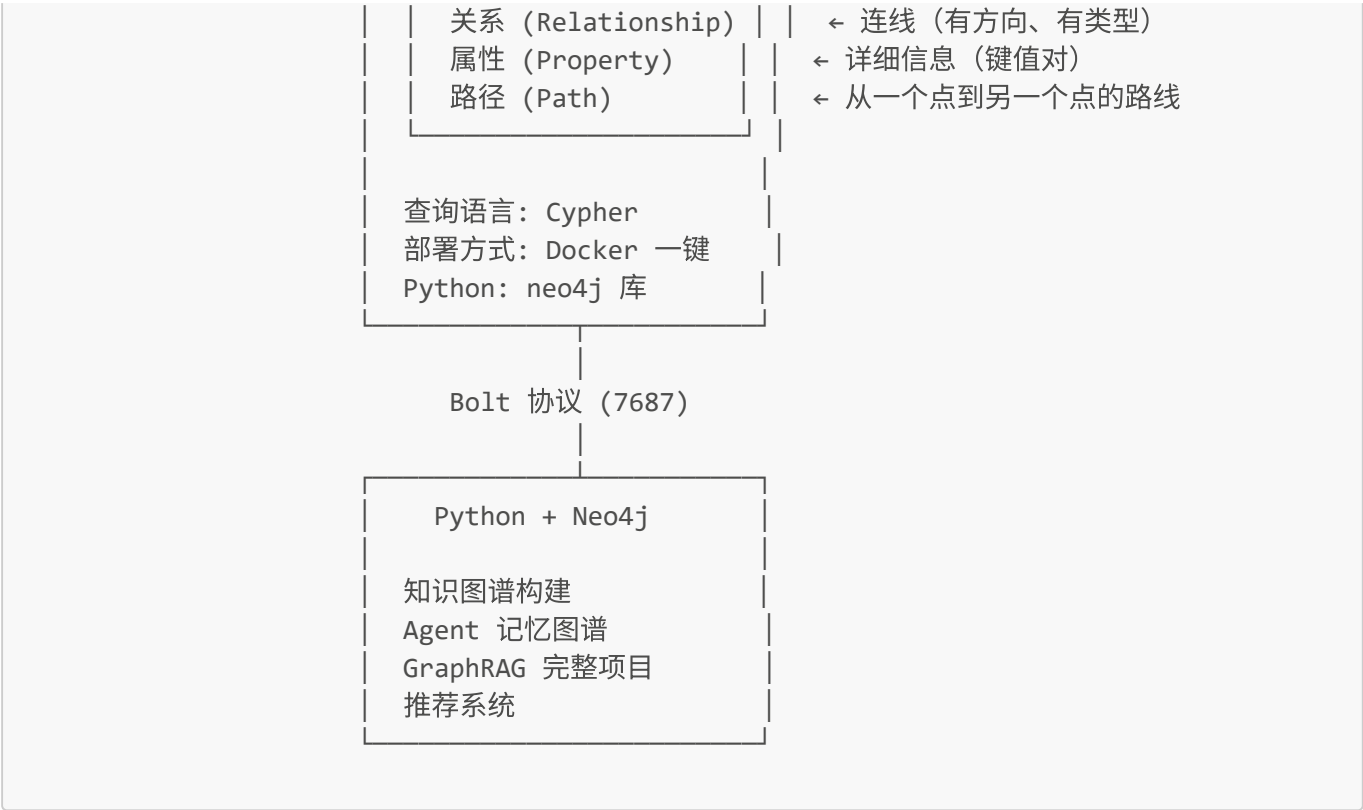
常见问题

- ▶ **？ Neo4j 容器启动后 Browser 打不开 (http://localhost:7474) ？**
- ▶ **？ Python 连接 Neo4j 报 "Connection refused" 或 "ServiceUnavailable"?**
- ▶ **？ Neo4j 容器内存占用过大怎么办？**
- ▶ **？ `MATCH (n)` 查出来什么都没有？**
- ▶ **？ `MERGE` 和 `CREATE` 的区别是什么？**
- ▶ **？ Cypher 查询很慢怎么办？**
- ▶ **？ 怎么删除 Neo4j 里的所有数据？**
- ▶ **？ `LOAD CSV` 报错 "file not found"?**
- ▶ **？ Neo4j 密码忘了怎么办？**
- ▶ **？ Python neo4j 库报 "ResultConsumedError"?**
- ▶ **？ Neo4j 和 GraphRAG 的关系——一定要用 Neo4j 吗？**

总结

核心概念回顾





关键点回顾

概念	一句话记住
为什么用图数据库	关系密集型数据（社交、推荐、知识图谱）用图比用表快 100 倍
图 vs 关系表	Excel 填表 vs 思维导图画图——图更直观
核心概念	节点=点、关系=线、属性=标签、路径=路线
Cypher	用 ASCII 艺术画关系——(a)-[:R]->(b)
Docker 部署	一条 docker run 拉起，7474（Browser）+ 7687（Bolt）
Python 连接	neo4j 库，bolt://localhost:7687
GraphRAG	知识图谱 + LLM——先查关系再回答，比纯语义检索更准确

与后续课程的衔接

你已经完成了 Neo4j 的"地基 + GraphRAG 实战"部分。接下来的课程会这样用到 Neo4j：

后续课程	会用到什么	你现在需要掌握什么
Agent 开发	记忆图谱、经验复用	理解图结构存记忆 vs List 存记忆的区别
RAG 系统升级	从 RAG 到 GraphRAG	完整 GraphRAG 项目模板直接复用
知识图谱项目	实体提取、关系建模	理解节点/关系/属性的建模方式
项目实战	完整 AI 后端架构	MySQL + Redis + Neo4j + 向量数据库的组合

🚀 **不用慌**：每个后续课程开头都会复习需要用到的 Neo4j 操作和 Cypher 语法。你现在只要确保**三阶段目标达成**——能部署、能用 Cypher 查询、能在 AI 项目中实战应用 GraphRAG。

🎯 **下一步**：确认 Docker Neo4j 启动成功、Browser 能访问、Python 能连接、完成 GraphRAG 项目运行后，进入下一阶段的课程学习。记住你的节奏：**部署** → **Browser 练习 Cypher** → **Python 连接** → **知识图谱** → **GraphRAG 项目**，这就是你的路径。